Problema 1: Sala de emergencias

La sala de emergencias prioriza la atención de pacientes según su **nivel de riesgo**:

- Nivel 1: críticos (prioridad máxima).
- Nivel 2: moderados.
- Nivel 3: bajos.

Si hay varios pacientes con el mismo nivel de riesgo, se atienden en **orden de llegada**. Esto se logra utilizando una **cola de prioridad implementada con un montículo binario mínimo**, donde los elementos con la clave más pequeña tienen mayor prioridad.

Implementación:

- **Clases principales:**
    1. `MonticuloBinario`: Representa el montículo binario.
        - Se implementa como una lista que inicia con un elemento "infinito negativo". Este elemento facilita los cálculos de posicionamiento en la estructura, ya que el movimiento por la lista se basa en divisiones o multiplicaciones por 2.
    2. `Paciente`: Modela los datos de cada paciente, incluyendo:
        - Nombre y apellido.
        - Nivel de riesgo.
        - Descripción del nivel de riesgo.
        - Número de orden (según el orden de llegada).

Funcionamiento:

1. Los pacientes se ingresan en el montículo según su nivel de riesgo y orden de llegada.
2. La sala muestra la lista de pacientes desordenada inicialmente, pero los organiza dinámicamente al momento de atender.
3. Entre cada atención se añaden pausas para simular una sala de emergencias real.

Problema 2: Registro de temperaturas

Se desarrolla un sistema para registrar y gestionar temperaturas en un rango de fechas, utilizando un **árbol AVL**. Este árbol mantiene un equilibrio constante para optimizar las operaciones.

Clases principales:

1. **`Arbol_AVL`**:
   - Representa el árbol binario equilibrado.
   - Cada nodo contiene:
     - Fecha (clave).
     - Temperatura (valor).
     - Factor de equilibrio.
     - Referencias al padre, hijo izquierdo y derecho.
   - Garantiza que las inserciones, eliminaciones y búsquedas se realicen en **O(log n)**.
2. **`Temperaturas_DB`**:
   - Gestiona el árbol AVL para almacenar y trabajar con las temperaturas.
   - Métodos principales:
     - **`guardar_temperatura`**: Añade una temperatura con su fecha.
     - **`devolver_temperatura`**: Busca la temperatura de una fecha específica.
     - **`max_temp_rango`** / **`min_temp_rango`**: Encuentra la mayor o menor temperatura en un rango de fechas.
     - **`temp_extremos_rango`**: Devuelve las temperaturas máxima y mínima en un rango.
     - **`borrar_temperatura`**: Elimina una temperatura por fecha.
     - **`temperaturas_en_rango`**: Lista las temperaturas registradas en un rango de fechas.

Complejidad de los métodos:

Método	Complejidad
guardar_temperatura	O(log n)
devolver_temperatura	O(log n)
max_temp_rango / min_temp_rango	O(k + log n)
borrar_temperatura	O(log n)
temperaturas_en_rango	O(k + log n)
temp_extremos_rango	O(k + log n)
recorrer_rango	O(k + log n)
devolver_temperaturas	O(k + log n)
cantidad_muestras	O(1)
nodos_en_rango O	(k + log n)

Donde *n* es el número total de temperaturas en el árbol, y *k* es el número de nodos en el rango de fechas.

Funcionamiento:

1. Las temperaturas se guardan con su fecha correspondiente.
2. Permite consultar datos específicos o analizar rangos, obteniendo información como los extremos de temperatura o eliminando registros.
3. El uso del árbol AVL asegura un acceso rápido y eficiente.

Problema 3: Ruta de mensajes entre aldeas

Este problema busca determinar la forma más eficiente de enviar un mensaje entre aldeas conectadas por caminos, evitando pasar por la misma aldea dos veces. Se utiliza el **algoritmo de Prim**, que construye un **árbol de expansión mínima (AEM)** basado en grafos.

Clases principales:

1. `Vértice`:
    - Representa cada aldea (nodo del grafo).
    - Almacena:
        - Conexiones con otras aldeas (aristas) y sus pesos.
        - La distancia mínima hasta otro nodo.
        - Su predecesor (nodo desde el cual se llega con menor costo).
2. `Grafo`:
    - Modela el conjunto de aldeas y sus conexiones.
    - Permite:
        - Añadir vértices y aristas.
        - Cargar datos masivamente para grafos grandes.

Funcionamiento del algoritmo Prim:

1. **Inicialización**:
    - Se define un vértice inicial (ej.: aldea "Peligros").
    - Todos los vértices comienzan con una distancia infinita y sin predecesores.
2. **Selección de rutas**:
    - Se utiliza el **montículo binario** para priorizar vértices según su distancia mínima.
    - En cada iteración, se selecciona el vértice con menor costo y se actualizan las distancias de sus vecinos.
3. **Resultado**:
    - El algoritmo construye un árbol de expansión mínima, que representa la ruta más eficiente para conectar todas las aldeas.

Ventajas de esta implementación:

- Utiliza el montículo binario previamente creado para optimizar la selección del vértice con menor distancia.
- Permite trabajar con grafos grandes gracias a su estructura modular.